DIGITAL ASSIGNMENT 3 — REVIEW 3

FinEdge

Personal Finance Management Web Application
Project Report

Course	Software Engineering
Author	vehladkid (GitHub)
Repository	https://github.com/vehladkid/FInEdge
Release	v1.0.0

Contents

1. Problem Statement
2. User Stories
3. System Architecture and System Design
4. Design of Tests
5. Appendix
    - A. Software Requirements Specification (SRS)
    - B. Data Flow Diagrams (DFD)
    - C. Entity-Relationship Diagram (ERD)
    - D. UML diagrams
    - E. Code listing / GitHub link

1. Problem Statement

Background

Most people can say roughly what they earn but not where it goes. Keeping track normally means either a spreadsheet — which needs manual formulas, has no notion of budgets that update themselves, and is awkward on a phone — or a commercial finance app that can be bloated, paid, or tied to a bank connection the user may not want. A student or young professional with a modest, irregular income needs something lighter: a place to write down each income and expense, see a total at a glance, and be told when a category is getting out of hand.

Problem

There is a need for a small web application that lets an individual:

1. **Authenticate securely**, so that financial records are private to their owner.
2. **Record income and expenses** quickly, and correct or remove them later.
3. **Organise transactions into categories**, including categories of their own.
4. **Set budgets** per spending category and period, and know at any moment how much of each is used — based on what was actually recorded, not on a number typed in by hand.
5. **Monitor financial performance** through a dashboard (balance, month totals, recent activity, budget status).
6. **Analyse trends** — income versus expenses over time, spending by category, daily spending, savings rate.
7. **Receive financial insights** in plain language: unusual spending, budgets approaching or exceeding their limit, overspending, and the savings rate.

Constraints that shape the solution

- **Scope.** A single-currency, single-user-per-account system. There are no bank integrations, multi-currency conversion, shared accounts, microservices or message queues; the design is a monolithic REST API with one relational database.
- **Correctness of money.** Amounts are stored as exact decimals (`NUMERIC(12,2)`), must be positive, and budget figures are derived on the server from the transaction table so that the dashboard, the budget screen and the insights can never disagree.
- **Privacy and security.** Passwords are stored only as bcrypt hashes; sessions are signed JWTs; every query is scoped to the authenticated user so one user can never read or alter another's data. The optional AI feature receives only aggregated monthly figures — never names, e-mail addresses or individual transactions — and the application must work fully without it.
- **Usability.** The interface must work on a phone as well as a desktop, and every screen must show a clear loading, error and empty state rather than invented values.
- **Maintainability and quality.** The system is built to be tested: business rules are pure functions covered by unit and mutation tests; the API is covered by integration tests against a real database; user journeys are covered by browser regression tests; and

the whole pipeline runs in CI.

Objective

Deliver FinEdge: a React single-page application backed by an Express API and a PostgreSQL database that satisfies the needs above, reproducibly deployable with Docker, and verified by the automated tests described in this report.

Out of scope

Bank synchronisation, receipts/attachments, recurring transactions, data export, password reset by e-mail, multi-currency, and mobile native apps.

2. User Stories

Each story is mapped to the feature that implements it and to the automated tests that protect it. Test paths are relative to `backend/` (`B:`) or `frontend/` (`F:`).

ID	User story	Implementation	Tests
US-01	As a user, I want to **register an account**, so that I can securely manage my financial data.	`POST /api/auth/register`, `pages/Register.jsx`, bcrypt hashing, 9 default categories	B: `integration/auth.test.js` (register, duplicate, validation, race); F: `unit/auth.test.jsx`; E2E `auth.spec.js`
US-02	As a user, I want to **log in**, so that my financial information is protected.	`POST /api/auth/login`, JWT, `AuthContext`	B: `auth.test.js` (valid, wrong password, unknown e-mail); F: `auth.test.jsx`; E2E `auth.spec.js`
US-03	As a user, I want protected pages to require login and **log out** when I finish, so that nobody else can open my data on a shared device.	`requireAuth`, `ProtectedRoute`, logout, expired-token handling	B: guard tests (no/invalid/expired/forged token); E2E: every protected route redirects, expired session
US-04	As a user, I want to **record an expense or income**, so that I can track where my money goes.	`POST /api/transactions`, `TransactionForm`, `AddTransaction`	B: `transactions.test.js`; F: `transactionForm.test.jsx`; E2E `transactions.spec.js`
US-05	As a user, I want to **edit, delete and filter** my transactions, so that I can correct mistakes and find entries.	`PUT/DELETE /api/transactions/:id`, filters + pagination, `Transactions` page	B: CRUD, filter, pagination tests; E2E: edit/delete, filter, paginate
US-06	As a user, I want to **categorise transactions** and manage my own categories, so that I can understand my spending patterns.	`/api/categories`, `Categories` page, delete protection	B: `categories.test.js`; E2E `categories-analytics.spec.js`
US-07	As a user, I want to **create a budget** for a category and period, so that I can control my spending.	`/api/budgets` (weekly/monthly/yearly), `Budget` page	B: `budgets.test.js`; F: `dashboard.test.jsx`; E2E `budgets.spec.js`; unit: `budgetCalc`, `dates`
US-08	As a user, I want my budget to show **spent, remaining, percentage and whether it is exceeded**, calculated from my real transactions.	SQL `spent` subquery + `budgetStatus()`	B: calculations (boundaries, other categories, other users); unit + mutation tests on `budgetCalc`
US-09	As a user, I want a **dashboard** with my balance, month totals, recent transactions and budget status, so that I see my position at a glance.	`GET /api/dashboard`, `Dashboard` page	B: `reports.test.js`; F: `dashboard.test.jsx`; E2E: empty state, totals

ID	User story	Implementation	Tests
US-10	As a user, I want **analytics** (income vs expenses, category breakdown, trends), so that I can understand my financial trends.	`GET /api/analytics`, Recharts	B: aggregation correctness; E2E: charts render, range control
US-11	As a user, I want **financial insights**, so that I can spot unusual spending or budget risks — with or without an AI service.	`GET /api/insights`, `insightRules`, `aiClient`	unit + mutation on `insightRules` / `aiClient`; B: rule-based fallback, AI path with injected provider, failure fallback; E2E: insights shown
US-12	As a mobile user, I want the app to be **usable on my phone**, so that I can add expenses wherever I am.	Responsive layout, collapsible navigation	E2E at 390×844: no horizontal overflow, navigation, forms, charts
US-13	As a user, I want clear **error, loading and empty states**, so that I always know what is happening and am never shown made-up numbers.	`Loading`, `ErrorMessage`, `Empty`, centralised API errors	F: dashboard/auth tests; E2E: API-unreachable state, empty states

3. System Architecture and System Design

1. Overview

FinEdge is a **layered monolith** deployed as three containers (frontend, backend, PostgreSQL) by `docker-compose.yml`. The brief for the project explicitly excludes microservices and queues; a single REST API over one relational database is sufficient for the data volume and keeps the system easy to run and test.

```
USER ─► React/Vite SPA ──HTTP JSON (Bearer JWT)──► Express API ──pg──► PostgreSQL
                                                        │
                        routes → middleware → controllers → services → repositories
                                                              └─ optional ─► AI provider
```

(Diagrams: `diagrams/uml-component.png`, `dfd-level-0.png`, `dfd-level-1.png`.)

2. Frontend (React 18, Vite, Tailwind 3, Recharts, react-router 7)

Concern	Where
Pages	`src/pages/` — Landing, Login, Register, Dashboard, Transactions, AddTransaction, Categories, Budget, Analytics
Layout & primitives	`layouts/PageShell`, `components/NavBar`, `components/ui/Card`, `ui/States` (loading/error/empty, shared field styles)
Session	`contexts/AuthContext.jsx` (user, status, login/register/logout) and `components/ProtectedRoute.jsx`; the JWT lives in `localStorage` through `services/session.js`, which falls back to memory if storage is blocked
Data access	`services/api.js` is the only module that calls `fetch`: base URL, bearer header, query params, uniform `ApiError`, and a `401` handler that clears the token and signs the user out. `services/finance.js` exposes one function per endpoint.
Async state	`hooks/useAsync.js` returns `{data, error, loading, reload}`
Charts	Recharts with shared tokens in `theme.js` (the same hex values feed Tailwind and the chart props)
Formatting	`utils/format.js` (currency, sign before symbol), `utils/dates.js`

The landing page's preview panel is a fixed, labelled *sample*; every authenticated screen shows only API data.

3. Backend (Express 4, Node 22)

```
src/
  app.js            createApp({ai}) — middleware + routes, importable without opening a port
  server.js         process entry point
  config/           the only reader of process.env
  routes/           api.js (auth guard applied after the public auth routes), health.js
  middleware/       auth (JWT), validate (zod -> req.valid), errorHandler
  controllers/      HTTP only: read req.valid, call a service, shape the response
  services/         business rules (auth, category, transaction, budget, analytics, insight)
                    pure logic: budgetCalc, analyticsCalc, insightRules   (no I/O)
                    aiClient: optional provider call
  repositories/     all SQL (parameterised); one file per table/report
  validators/       zod schemas + parse()
  db/               pool.js (type parsers), migrate.js, migrations/
  utils/            dates, password policy, money rounding, clock
  errors/           AppError and factories
```

Authentication. `register` checks for an existing e-mail, hashes with bcrypt, inserts the user and the nine default categories in one transaction, and returns a JWT (`sub` = user id). `login` compares the hash and answers identically for unknown e-mail and wrong password. `requireAuth` verifies the bearer token and sets `req.userId`; services never trust an id from the client.

Request flow. `route → requireAuth → validate(schema) → controller → service → repository → PostgreSQL`; any thrown `AppError` (or unexpected error) reaches `errorHandler`, which emits `{success:false, error:{code,message,details?}}` and hides internals in production.

Budgets. `repositories/budgets.js` selects each budget together with `spent`, a correlated subquery summing the user's expense transactions in that category between `start_date` and `end_date`. `services/budgetCalc.budgetStatus` turns `(amount, spent)` into `remaining`, `percentUsed`, `exceeded` and `status`. The same function feeds the budget list, the dashboard and the insights.

Analytics. `analyticsService` issues aggregate queries (totals by type, monthly totals, expenses by category, daily expenses) and `analyticsCalc` zero-fills months and computes shares; no endpoint ships raw transaction lists for charting.

Insights. `insightService` gathers aggregates for this and the previous month and calls the provider if `AI_API_KEY` is set. Any failure, empty answer or missing key falls through to `insightRules.generateInsights`, a pure function. Only aggregated figures — no names, e-mails or individual rows — are ever sent to the provider.

4. Database

PostgreSQL 16; schema in `backend/src/db/migrations/001_init.sql` (see ERD). Design choices: `NUMERIC(12,2)` for money with `CHECK (amount > 0)`; `ON DELETE RESTRICT` from transactions to categories (a label cannot vanish under a record) and `CASCADE` to budgets; unique `lower(email)`; unique `(user_id, name, type)` categories and `(user_id, category_id, period, start_date)` budgets; indexes on `(user_id, transaction_date DESC)` and `category_id`. The `pg` type parsers return numbers for `NUMERIC` and plain strings for `DATE`, so no timezone shifting occurs. **No other datastore (cache, queue, search) is used.**

5. Deployment and configuration

- `docker-compose.yml`: `postgres` (16, health-checked), `backend` (runs `node src/db/migrate.js && node src/server.js`), `frontend` (Vite dev server on 5173).
- Configuration is by environment variables (README). Secrets are never committed; `docker-compose.yml` contains only a labelled local-development JWT secret default.
- No cloud deployment (e.g. Vercel) is configured in this repository. For production, serve `frontend/dist` from a static host or reverse proxy and set `JWT_SECRET`, `CORS_ORIGIN`, `VITE_API_URL`.
- CI (`.github/workflows/ci.yml`): backend (lint, unit, integration with a PostgreSQL service, mutation), frontend (lint, unit, build), end-to-end (Playwright, PostgreSQL service) and Docker image builds.

6. Quality attributes and how the design delivers them

Attribute	Mechanism
Security	bcrypt, JWT, per-user scoping in every query, parameterised SQL, validation at the edge, body size limit, no stack traces in production
Consistency	budget/dashboard/insights share one calculation path; totals computed from rows, never stored
Testability	`createApp()` has no side effects; pure modules; injectable AI provider; isolated `*_test` database guard
Resilience	optional AI never blocks insights; storage-blocked sessions fall back to memory; API errors become readable UI messages

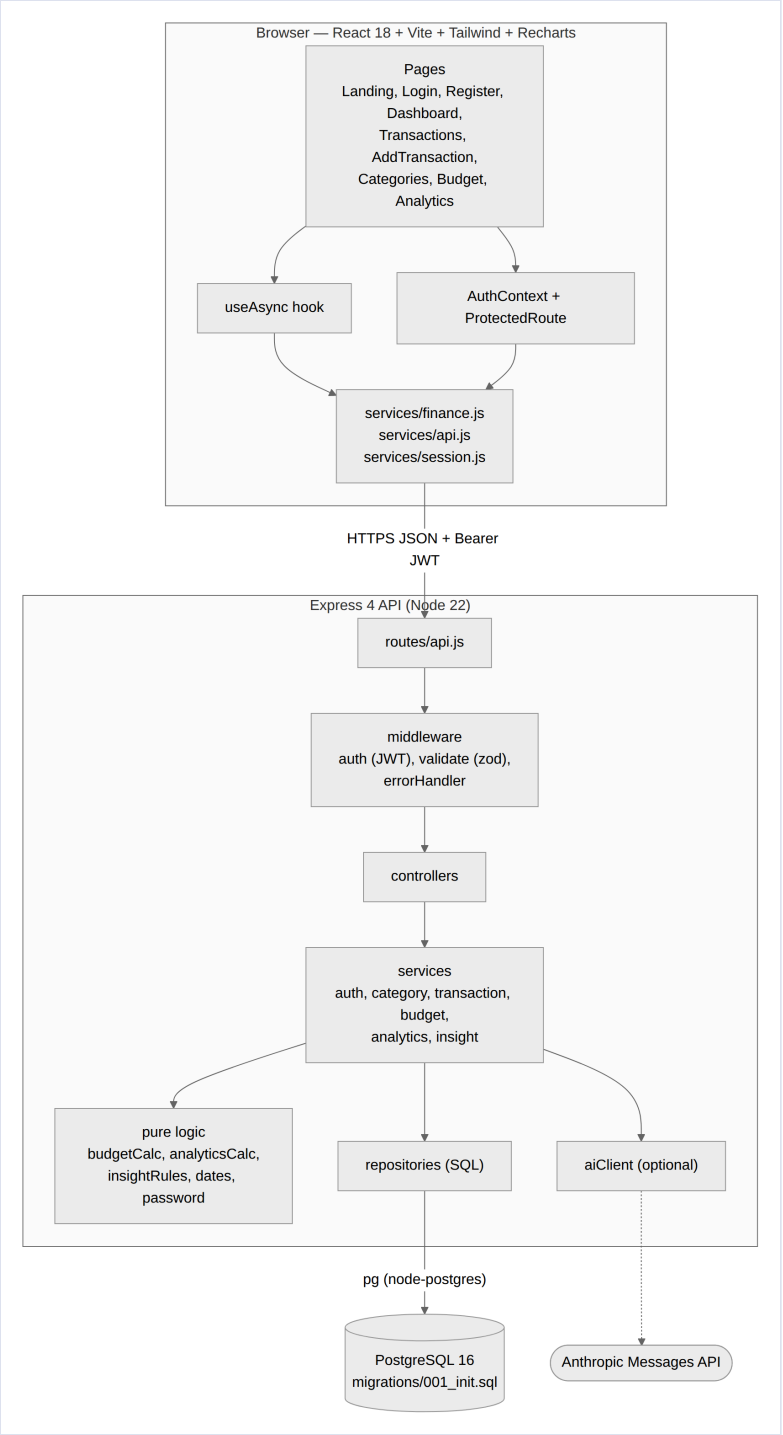


Figure 3.1 — Component view of the implemented system (see also Appendix D).

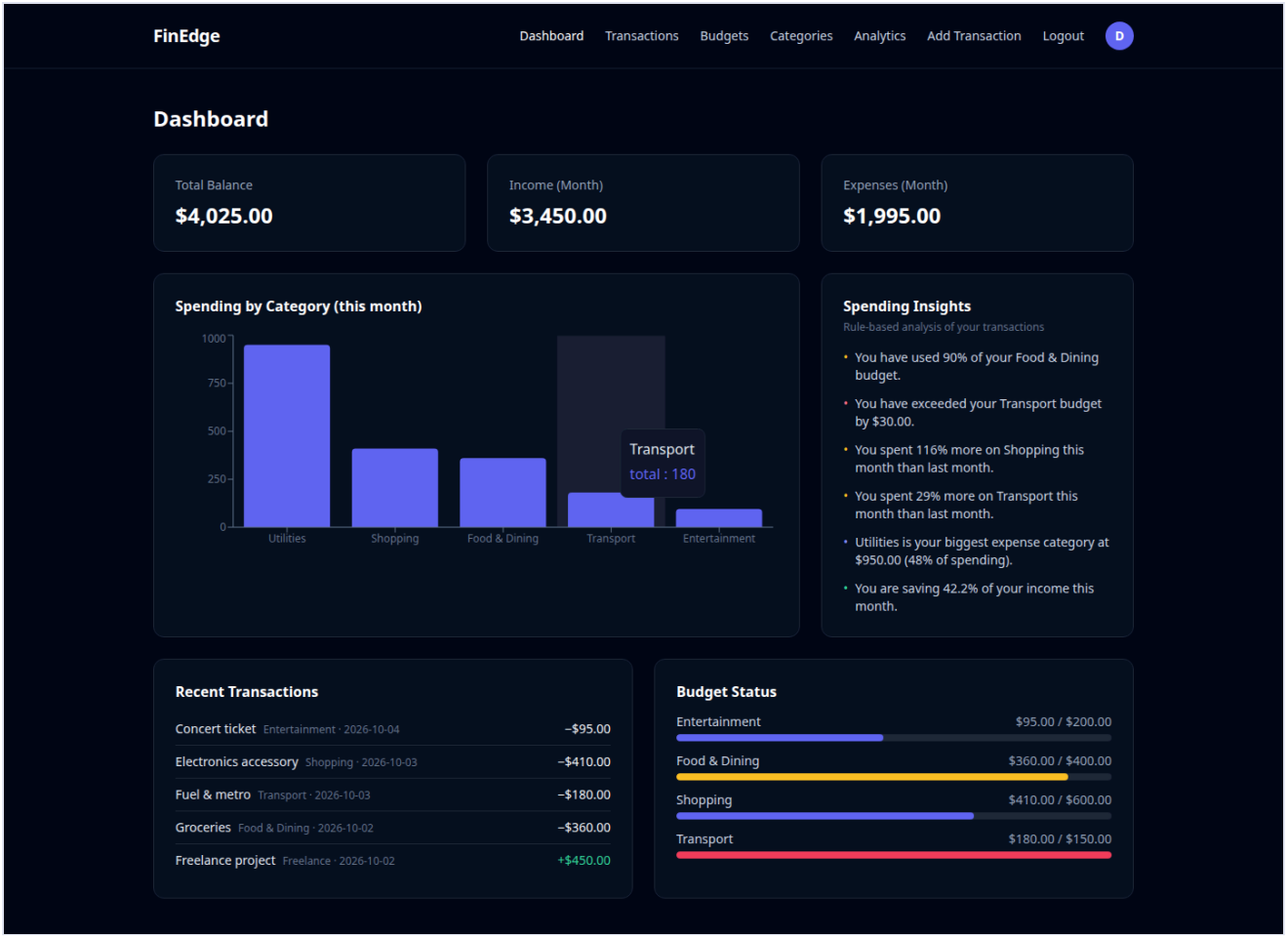


Figure 3.2 — The implemented dashboard (real data from PostgreSQL).

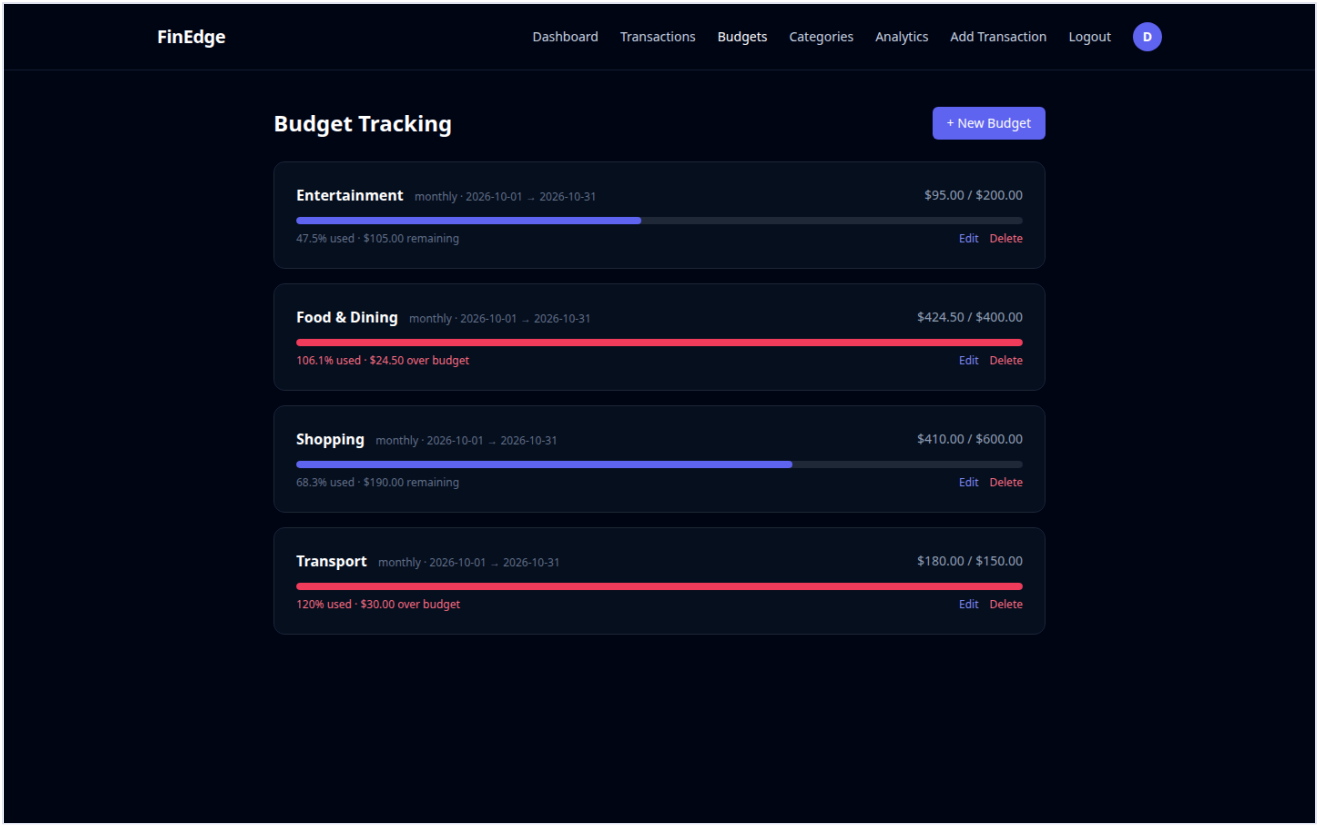


Figure 3.3 — Budgets with backend-calculated spent, remaining and status.

4. Design of Tests

1. Strategy

Testing is organised as a pyramid, and each layer targets a different kind of defect:

Layer	Tooling	Target	Count (v1.0.0)
Unit — backend	Vitest	Pure business logic: validators, dates, password policy, money rounding, budget/analytics calculations, insight rules, AI client parsing, error handler	135 tests, 9 files
Unit / component — frontend	Vitest, jsdom, React Testing Library, user-event	`formatCurrency`, API client, session storage, auth flow & protected route, transaction form, dashboard and budget pages	39 tests, 5 files
Integration	Vitest + Supertest, real PostgreSQL (`finedge_test`)	The real Express app end to end through routes, validation, services, SQL and the database	75 tests, 5 files
Regression / end-to-end	Playwright (Chromium)	Real browser → real API → real PostgreSQL (`finedge_e2e`) at 1440×900, 1280×800 and 390×844	40 tests × 3 viewports = 120
Mutation	StrykerJS (Vitest runner)	The quality of the unit tests on the business-logic modules	547 mutants, score 96.53 %

2. Principles

- **Test behaviour, not implementation.** Unit tests assert inputs and outputs (and, for the API, status codes, bodies and database effects); they do not inspect private state.
- **Real dependencies where the risk is.** Integration tests use a real PostgreSQL database — SQL, constraints, transactions and aggregation are exactly where mocks would hide bugs. Only external services are replaced: the AI provider is injected (`createApp({ ai })`) and its `fetch` is a parameter.
- **Isolation.** The integration setup refuses to run unless the database name ends with `_test` and truncates tables before each test; each E2E test registers its own user so tests are independent and can run in parallel.
- **Boundaries first.** Cases are chosen at the edge of every rule: a password of 7/8/72/73 characters, a budget exactly at 80 % / 100 % / one cent over, the first and last day of a budget period, a date range with `from == to`, amounts of 0.01 and 9 999 999 999.99, February 29/30, empty lists, zero income, zero expenses.
- **Security cases are first-class.** Missing/invalid/expired/forged tokens, wrong password vs unknown e-mail (identical responses), duplicate e-mail under concurrent requests, and every cross-user access path (read, edit, delete, use another user's category, budget spend leakage).
- **No fabricated baselines.** The regression suite asserts concrete behaviour (text, totals, URLs, element counts, absence of overflow) rather than comparing against auto-generated screenshots, so a pass means the behaviour is right, not merely unchanged.

3. Test design by feature

Authentication (integration, unit, E2E)

Equivalence classes: valid registration; each invalid field (name, e-mail, password rule); duplicate e-mail (any case; simultaneous); valid login; wrong password; unknown e-mail; no token; wrong scheme; empty bearer; garbage token; expired token; token signed with another secret; token for a deleted user. UI: success redirect (to the originally requested page), error messages, button disabled while submitting, token cleared when rejected, session expiry mid-use.

Transactions

CRUD happy paths; each validation rule (amount ≤ 0, text amount, bad type, impossible date, description > 200); unknown or other-user category; category/type mismatch; filters alone and combined; inclusive date bounds; pagination first/last/beyond-last; isolation between users (get/update/delete/create-with-foreign-category).

Categories

List and type filter (including an ignored invalid filter); create with trimming; duplicates (same type → 409, other type → allowed); rename and rename-collision; delete unused, blocked when used (singular/plural message), allowed again after the transactions go; budgets removed with their category; isolation.

Budgets

Creation derives the end date; duplicates and collisions on update; only expense categories; `spent` counts only this category, only expenses, only this user, only inside the inclusive period; `ok` → `warning` at exactly 80 % → exactly-at-budget (100 %, *not* exceeded) → one cent over (*exceeded*); edits and deletes of transactions change the budget immediately. Pure `budgetStatus` and `endOfPeriod` (week/month/year, month-end clamping, leap years) are unit- and mutation-tested.

Reports and insights

Dashboard on an empty account (all zeros, no invented data) and with data (balance across months, shares, active budgets only); analytics zero-fill, window exclusion, daily series, range validation; insights: empty month, each rule at and just below its threshold, rule ordering, tie-breaking, rule-based fallback, AI success, AI failure/empty/invalid answers, and that nothing resembling an e-mail or password is sent to the provider.

Responsive and resilience (E2E)

Every page at three viewports has a visible heading, finished loading and `scrollWidth ≤ clientWidth`; navigation works with and without the mobile menu; charts render (`svg`, pie sectors, bars); dialogs open and close; every page loads with **no console errors, page errors, failed or ≥ 400 requests**; the API being unreachable produces a readable error and a working Retry.

4. Mutation testing design

Mutation testing is applied to the modules that contain decisions and arithmetic and have no I/O: `utils/dates`, `utils/password`, `utils/money`, `validators`, `services/budgetCalc`, `analyticsCalc`, `insightRules`, `aiClient`, `middleware/errorHandler`, `errors`. Animation-heavy React components and SQL repositories are deliberately not mutated; they are covered by the browser and integration tests. The build fails below 80 %. Results and the rationale for each surviving mutant are in `TESTING.md`.

5. Entry/exit criteria

A change is releasable when lint is clean, all unit, integration and regression tests pass, the mutation score is ≥ 80 %, the production build and both Docker images build, and CI is green.

4.6 Results (v1.0.0)

All numbers below come from the unedited logs in `evidence/logs/` (produced by `tools/run-evidence.sh` from clean `npm ci` installs). Design rationale is in `design-of-tests.md`.

Commands

Command	Directory	Result
`npm run lint`	backend, frontend	clean (0 errors, 0 warnings)
`npm run test:unit`	backend	**135 passed** (9 files)
`npm run test:unit`	frontend	**39 passed** (5 files)
`npm run test:integration`	backend	**75 passed** (5 files, real PostgreSQL)
`npm run test:mutation`	backend	**96.53 %** (threshold 80 %)
`npm run test:regression`	frontend	**120 passed** (40 tests × 3 viewports)
`npm run build`	frontend	success (see warnings)
`docker build` backend / frontend	repo root	both images build

`npm run test:e2e` runs the same Playwright suite at 1440×900 only; `test:regression` runs all three viewports.

Integration testing

`backend/tests/integration/*.test.js` drive the real Express app with Supertest against PostgreSQL. `setup.js` migrates the schema and refuses to run on a database whose name does not end in `_test`. Files: `auth`, `transactions`, `categories`, `budgets`, `reports` (dashboard, analytics, insights). Covered: every endpoint's success path; validation failures; 401/404/409 responses; cross-user isolation; database effects (hashed password, cascades, restrict); budget arithmetic against real rows; aggregation correctness; AI provider injected as success / failure / empty with a check that no e-mail or password-like text is sent; concurrent duplicate registration.

Regression testing

The Playwright suite (`frontend/tests/e2e`) runs the **real** React app, API and PostgreSQL (`finedge_e2e`) and is the regression net for user journeys:

Spec	Journeys
`auth.spec.js`	register → empty dashboard → logout; login returns to requested page; wrong password; server validation; duplicate e-mail; all protected routes redirect; invalid stored token; session expiring mid-use
`transactions.spec.js`	add expense (appears in list and dashboard); add income; client validation; edit and delete; filters; pagination; API-unreachable error + Retry; navigation
`budgets.spec.js`	empty → create; spent/remaining/percent from real transactions and after adding one through the UI; edit/delete; duplicate budget; validation; dashboard budget status and insight
`categories-analytics.spec.js`	create/rename/use/delete-blocked/delete category; duplicate; analytics empty state, totals and charts, range control; rule-based insights
`layout.spec.js`	six authenticated pages + three public pages: heading visible, loading finished, **no horizontal overflow**; navigation (collapsible on mobile); charts; dialogs; add-transaction usable
`health.spec.js`	every page loads with no console errors, page errors or failed/≥400 requests; landing links and backend status

Viewports: **Desktop 1440×900, Laptop 1280×800, Mobile 390×844** (touch). The strategy deliberately avoids pixel-diff baselines: a baseline generated from the same build proves nothing, so assertions check behaviour (text, numbers, URLs, element counts, overflow). Screenshots of the UI (`screenshots/app/`) are captured separately for the documents.

Real defects found by this suite during development and fixed: a table's screen-reader-only cell caused 16 px of horizontal scroll on mobile; currency rendered negative values as `$-42.10`; a stale in-memory token survived `localStorage` being cleared.

Mutation testing

- **Tool:** StrykerJS 10, Vitest runner, `coverageAnalysis: perTest` (`backend/stryker.config.json`). The runner requires Vitest ≤ 3.x here (with Vitest 5 every mutant survived, which is how the incompatibility was detected), so Vitest is pinned to 3.
- **Targets (10 files):** `utils/dates`, `utils/password`, `utils/money`, `validators/index`, `services/budgetCalc`, `analyticsCalc`, `insightRules`, `aiClient`, `middleware/errorHandler`, `errors/index`. Nothing was excluded to improve the score; UI components and SQL repositories are covered by the E2E and integration layers instead.
- **Result:** 547 mutants — **527 killed, 1 timeout (detected), 19 survived**; score **96.53 %**; per-file table and every survivor in `evidence/mutation-summary.md`.
- **Process:** first run 17.85 % exposed the Vitest 5 incompatibility; after pinning, 94.41 % → tests were added for each real gap (error messages, ties, severities, trimming, prompt shape, boundary dates) → redundant code was removed where a guard was dead (`isValidIsoDate` now round-trips through `Date`) → 96.53 %.

Surviving mutants and why they were not "fixed"

Mutants	Reason
`aiClient.js:31` (3), `:38` (1)	Provider-body shape variants all end in the same observable result (a thrown error → rule-based fallback); the severity default equals the literal that was blanked. Equivalent.
`insightRules.js:54` (2), `:66` (2), `:104` (1)	Guards that are unreachable with consistent data (categories exist only when expenses > 0; at equality the growth is 0 %, below the 20 % threshold; income 0 with no insight cannot occur once savings rules fire). Equivalent.
`dates.js:4` (2), `:7` (3)	Defence-in-depth guards (anchors, `typeof`) made redundant by the final round-trip check `toIso(date) === value`, which rejects those inputs anyway.
`validators:11` (1), `:25` (1), `:39` (1)	`join(".")` only matters for nested field paths (none exist); `<` vs `<=` at a 1e-6 floating-point epsilon is unreachable; zod's issue `code` is metadata.
`validators:48`, `:75` (2)	Replacing a schema with `{}` makes the module throw on import (`.pick`/`.omit` of a missing key), so the test files fail to load — detected by the suite, but Stryker's runner reports a load failure as *survived*.

Coverage

Scope	Statements	Branches	Functions
Backend, unit tests (logic modules, error handler, AI client)	99.22 %	99.22 %	96.55 %
Backend, integration tests (all `src/`, excluding `server.js` and `migrate.js`)	90.94 %	90.28 %	99.08 %
Frontend unit tests (components, contexts, hooks, services, utils)	95.78 %	87.5 %	49.01 %

Not covered by unit coverage, by design: `frontend/src/pages/*` and the charts (covered by Playwright — Recharts needs real layout, so unit tests stub it), `services/finance.js` (one-line wrappers, mocked in unit tests and exercised by every E2E test — which is why function coverage is 49 %), `server.js` (process start), `db/migrate.js` (run by every integration and E2E start-up and by Docker). Backend integration coverage gaps are the AI provider's network call (unit-tested with an injected `fetch`) and the production branch of the error handler (unit-tested).

Known warnings

- `vite build` warns that the JS bundle is 629 kB (> 500 kB) because Recharts is bundled in one chunk. Not an error; code-splitting is a possible future improvement.
- `npm audit` reports advisories in transitive development dependencies of the frontend toolchain; none are in the runtime path and they were not force-upgraded to avoid breaking changes.

Appendix

A. Software Requirements Specification

1. Introduction

1.1 Purpose

This document specifies the requirements of FinEdge, a personal finance management web application, as implemented in release v1.0.0. It is written for the course evaluators, developers and testers; every requirement below corresponds to behaviour that exists in the repository and, where stated, to an automated test.

1.2 Scope

FinEdge lets a registered user record income and expenses, organise them into categories, set budgets, and view a dashboard, analytics and insights. It consists of a React single-page application, an Express REST API and a PostgreSQL database. It does not integrate with banks, support multiple currencies, or provide recurring transactions or data export.

1.3 Definitions

Transaction — one income or expense record. **Category** — a user-owned label of type income or expense. **Budget** — a spending limit for one expense category over a weekly, monthly or yearly period. **JWT** — JSON Web Token used as the session credential.

1.4 Intended users

Individuals — typically students and young professionals — managing personal income and expenses. There is one user role: *registered user*. Unauthenticated visitors can see the landing page, register and log in.

2. Overall description

- **Product perspective:** a self-contained three-tier web system (browser → API → database), deployable with Docker Compose.
- **Operating environment:** modern evergreen browsers; Node.js 22; PostgreSQL 16.
- **Constraints:** single currency; monolithic REST API; no external service is required (the AI provider is optional).
- **Assumptions:** users have a modern browser with JavaScript; the server clock is correct (date ranges use UTC dates); PostgreSQL is available to the API.
- **Dependencies:** Express, `pg`, `bcryptjs`, `jsonwebtoken`, `zod`, React, Vite, Tailwind, Recharts, react-router; optional Anthropic Messages API.

3. Functional requirements

Authentication

ID	Requirement
FR-01	The system shall register a user from name (1–100 characters), e-mail (valid, ≤ 254) and password (8–72 characters with at least one letter and one digit).
FR-02	The system shall reject registration with an e-mail already in use, compared case-insensitively, including under concurrent requests (HTTP 409).
FR-03	The system shall store only a bcrypt hash of the password and shall never return it.
FR-04	The system shall authenticate by e-mail and password and return a signed JWT valid for 2 hours; an unknown e-mail and a wrong password shall produce the same 401 response.
FR-05	All routes other than registration, login and the health check shall require a valid bearer JWT; missing, malformed, expired or forged tokens shall yield 401.
FR-06	The UI shall redirect unauthenticated visitors from protected pages to the login page, return them to the requested page after login, and discard invalid or expired tokens.
FR-07	A new account shall receive nine default categories (six expense, three income).

Transactions

ID	Requirement
FR-08	The system shall create transactions with type (income/expense), amount (> 0, ≤ 2 decimals), category, date (valid `YYYY-MM-DD`) and optional description (≤ 200).
FR-09	The category of a transaction shall belong to the same user and have the same type as the transaction.
FR-10	The system shall read one transaction and list transactions newest first with pagination (default 20, max 100) and filters by type, category and inclusive date range; `from` shall not be after `to`.
FR-11	The system shall update and delete transactions.
FR-12	A user shall never read, modify or delete another user's transactions (responses are 404).

Categories

ID	Requirement
FR-13	The system shall list (optionally by type), create and rename categories; names are unique per user and type (409 on duplicates).
FR-14	The system shall refuse to delete a category used by transactions (409, with the count) and shall delete unused categories together with their budgets.

Budgets

ID	Requirement
FR-15	The system shall create, read, update and delete budgets (amount > 0; period weekly, monthly or yearly; start date). The end date shall be derived from the period; budgets may only target expense categories; a category, period and start date combination is unique.
FR-16	For every budget the system shall compute — in the database/backend — `spent` (sum of the user's expenses in that category between start and end dates inclusive), `remaining`, `percentUsed` (one decimal), `exceeded` (spent strictly greater than amount) and `status`: `ok`, `warning` (≥ 80 %) or `exceeded`.

Reporting

ID	Requirement
FR-17	`GET /api/dashboard` shall return the all-time balance, current-month income, expenses and savings rate, current-month spending by category with share, the five most recent transactions and the budgets active today.
FR-18	`GET /api/analytics?months=N` (1–24, default 6) shall return a zero-filled monthly income/expense/net series, spending by category, daily spending for the last 30 days and range totals.
FR-19	`GET /api/insights` shall return insights derived from the user's data: budget exceeded or near limit, category spend spikes (≥ 20 % and ≥ $20), a dominant category (≥ 40 %), month-over-month growth (≥ 20 %), overspending, and savings rate (< 10 % low, ≥ 20 % good).
FR-20	If an AI provider key is configured the system shall try the provider with aggregated figures only; on any failure or empty answer it shall return the rule-based insights. The response states its `source`.

User interface

ID	Requirement
FR-21	The UI shall provide pages for landing, registration, login, dashboard, transactions (list/edit/delete/filter), add transaction, categories, budgets and analytics.
FR-22	Each data page shall show loading, error (with retry) and empty states and shall not display invented values.
FR-23	The UI shall be usable at 390×844, 1280×800 and 1440×900 without horizontal scrolling, with a collapsible navigation on narrow screens.

4. External interfaces

- **User interface:** the pages listed in FR-21.
- **HTTP API:** JSON over HTTP; success `{ "success": true, "data", "meta?" }`; failure `{ "success": false, "error": { "code", "message", "details?" } }`. Routes are listed in the README.
- **Database interface:** PostgreSQL via `pg`; schema in `backend/src/db/migrations/001_init.sql`.
- **AI provider (optional):** Anthropic Messages API (`POST https://api.anthropic.com/v1/messages`).

5. Database requirements

Four tables — `users`, `categories`, `transactions`, `budgets` — with the keys, `CHECK` constraints and indexes shown in the ERD. Money is `NUMERIC(12,2)`. Deleting a user cascades to all their rows; deleting a category is restricted by transactions and cascades to budgets. The schema is created only by versioned migrations (`schema_migrations` records applied files).

6. Non-functional requirements

ID	Category	Requirement
NFR-01	Security	Passwords hashed with bcrypt (cost configurable, default 10); JWT signed with a server secret; the secret and AI key are read only from the environment and never reach the browser.
NFR-02	Security	Every data query is filtered by the authenticated user id; request bodies are limited to 100 kB; all input is validated before use; SQL uses parameters only.
NFR-03	Reliability	Errors return a consistent JSON shape; in production the cause of unexpected errors is never exposed.

ID	Category	Requirement
NFR-04	Performance	Dashboard and analytics are computed by database aggregation instead of transferring every transaction; lists are paginated; indexes cover user/date and category lookups. *(Not load-tested.)*
NFR-05	Usability	Responsive layout; labelled form controls; `role="alert"` / `status` messages; progress bars expose `aria-valuenow`.
NFR-06	Portability	`docker compose up --build` starts the whole system; migrations run on backend start and wait for the database.
NFR-07	Maintainability	Layered code; pure logic isolated; ESLint clean; CI on every push.
NFR-08	Testability	Unit, integration, regression (3 viewports) and mutation tests; see the Design of Tests.

7. Constraints

Single currency; UTC date arithmetic; password maximum of 72 characters (bcrypt input limit); the frontend Docker image runs the Vite dev server and is intended for development/demonstration rather than production serving.

8. Traceability

Requirement-to-test traceability is given in `user-stories.md` and `evidence-matrix.md`.

B. Data Flow Diagrams

Level 0 shows FinEdge as one process with the user, the PostgreSQL database and the optional AI provider. Level 1 decomposes it into the six processes implemented by the API (authenticate, manage categories, manage transactions, manage budgets, dashboard & analytics, generate insights) and the four data stores, which are the four database tables.

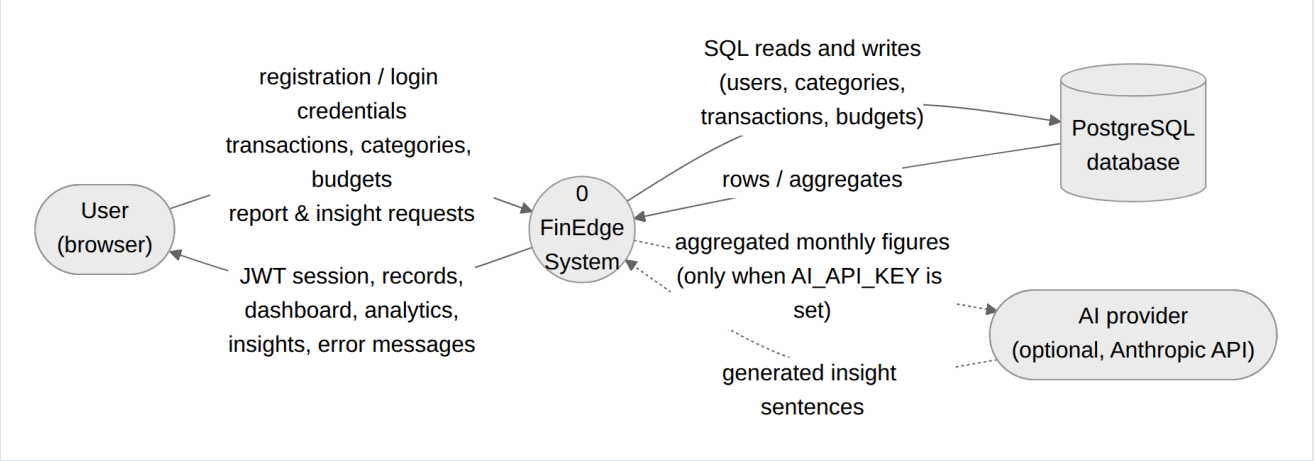


Figure B.1 — DFD level 0 (context diagram).

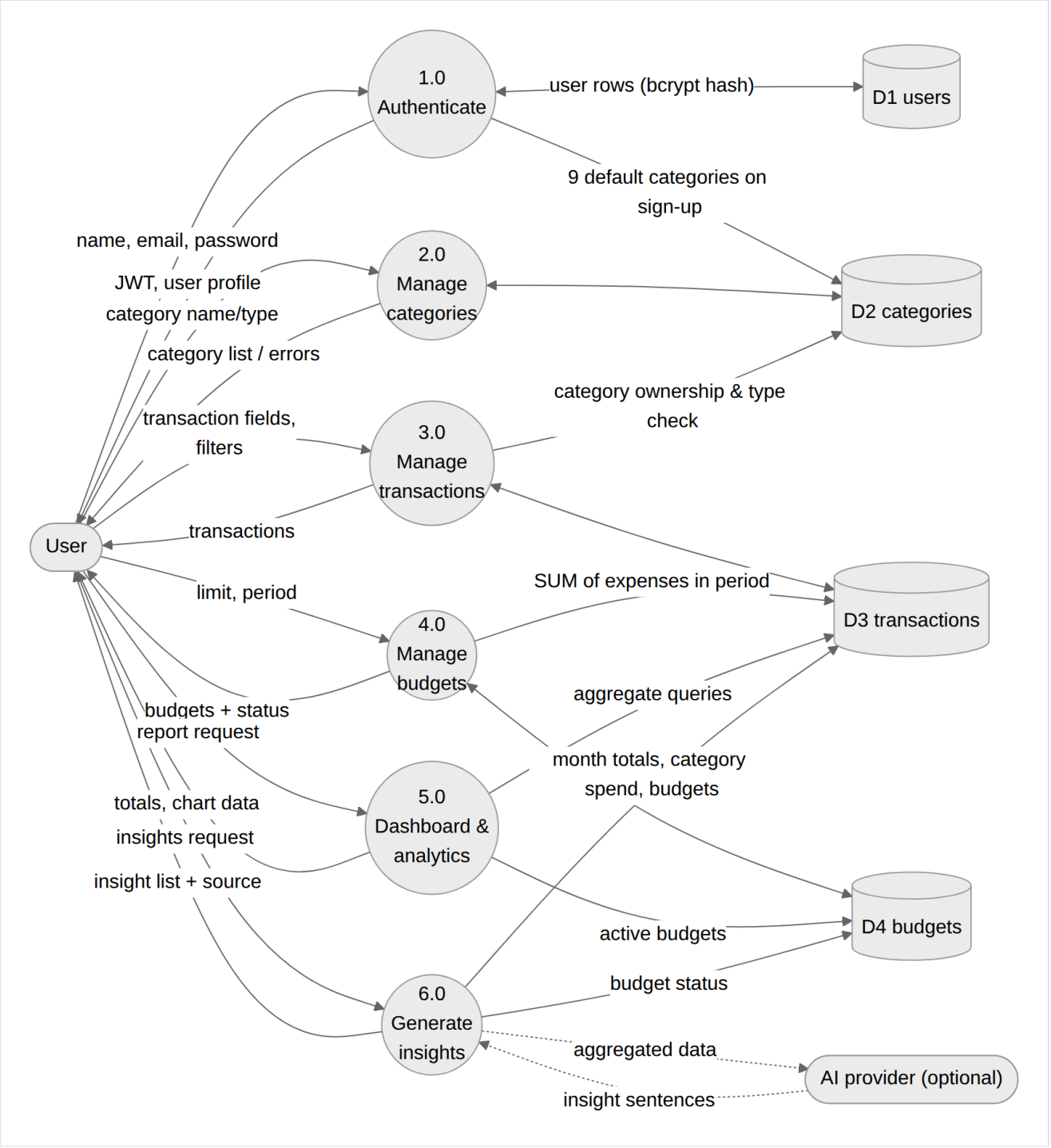


Figure B.2 — DFD level 1.

C. Entity-Relationship Diagram

The ERD mirrors `backend/src/db/migrations/001_init.sql`. PostgreSQL is the only datastore.

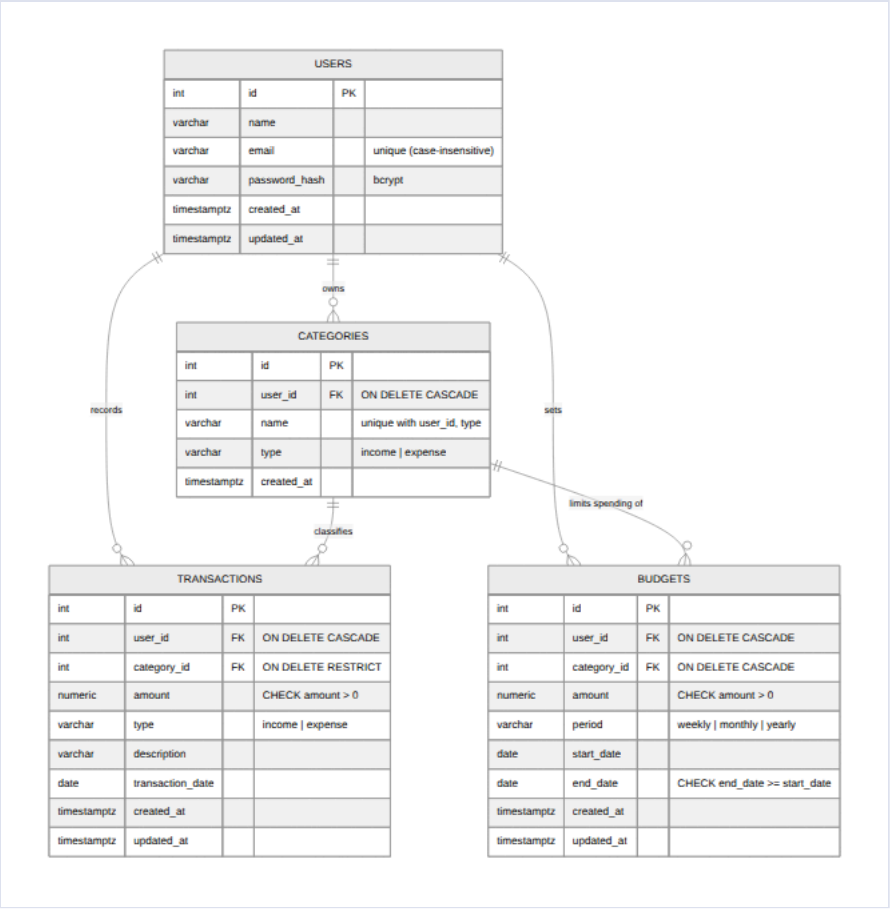


Figure C.1 — ERD of the implemented schema.

`backend/src/db/migrations/001_init.sql` — schema (excerpt) (lines 5–60)

```
CREATE TABLE users (
  id            SERIAL PRIMARY KEY,
  name          VARCHAR(100) NOT NULL,
  email         VARCHAR(254) NOT NULL,
  password_hash VARCHAR(100) NOT NULL,
  created_at    TIMESTAMPTZ  NOT NULL DEFAULT now(),
  updated_at    TIMESTAMPTZ  NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX users_email_unique ON users (lower(email));

CREATE TABLE categories (
  id         SERIAL PRIMARY KEY,
  user_id    INTEGER     NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name       VARCHAR(50) NOT NULL,
  type       VARCHAR(10) NOT NULL CHECK (type IN ('income', 'expense')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (user_id, name, type)
);
CREATE INDEX categories_user_idx ON categories (user_id);

CREATE TABLE transactions (
  id               SERIAL PRIMARY KEY,
  user_id          INTEGER       NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  category_id      INTEGER       NOT NULL REFERENCES categories(id) ON DELETE RESTRICT,
  amount           NUMERIC(12,2) NOT NULL CHECK (amount > 0),
  type             VARCHAR(10)   NOT NULL CHECK (type IN ('income', 'expense')),
  description      VARCHAR(200)  NOT NULL DEFAULT '',
  transaction_date DATE          NOT NULL,
  created_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
  updated_at       TIMESTAMPTZ   NOT NULL DEFAULT now()
);
CREATE INDEX transactions_user_date_idx ON transactions (user_id, transaction_date DESC);
CREATE INDEX transactions_category_idx ON transactions (category_id);

CREATE TABLE budgets (
  id          SERIAL PRIMARY KEY,
  user_id     INTEGER       NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  category_id INTEGER       NOT NULL REFERENCES categories(id) ON DELETE CASCADE,
  amount      NUMERIC(12,2) NOT NULL CHECK (amount > 0),
  period      VARCHAR(10)   NOT NULL CHECK (period IN ('weekly', 'monthly', 'yearly')),
  start_date  DATE          NOT NULL,
  end_date    DATE          NOT NULL,
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT now(),
  CHECK (end_date >= start_date),
  UNIQUE (user_id, category_id, period, start_date)
);
CREATE INDEX budgets_user_idx ON budgets (user_id);
```

D. UML Diagrams

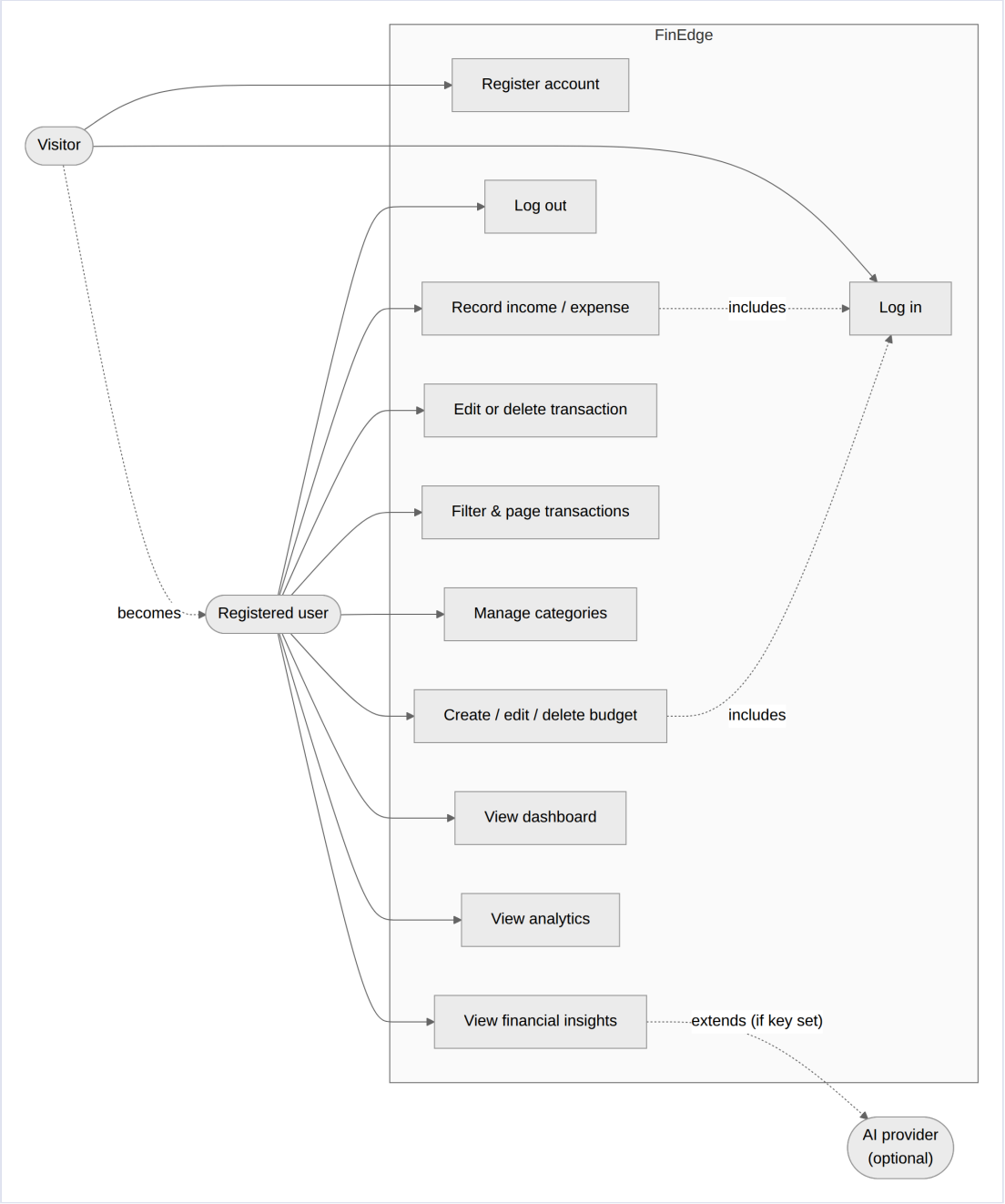


Figure D.1 — Use case diagram.

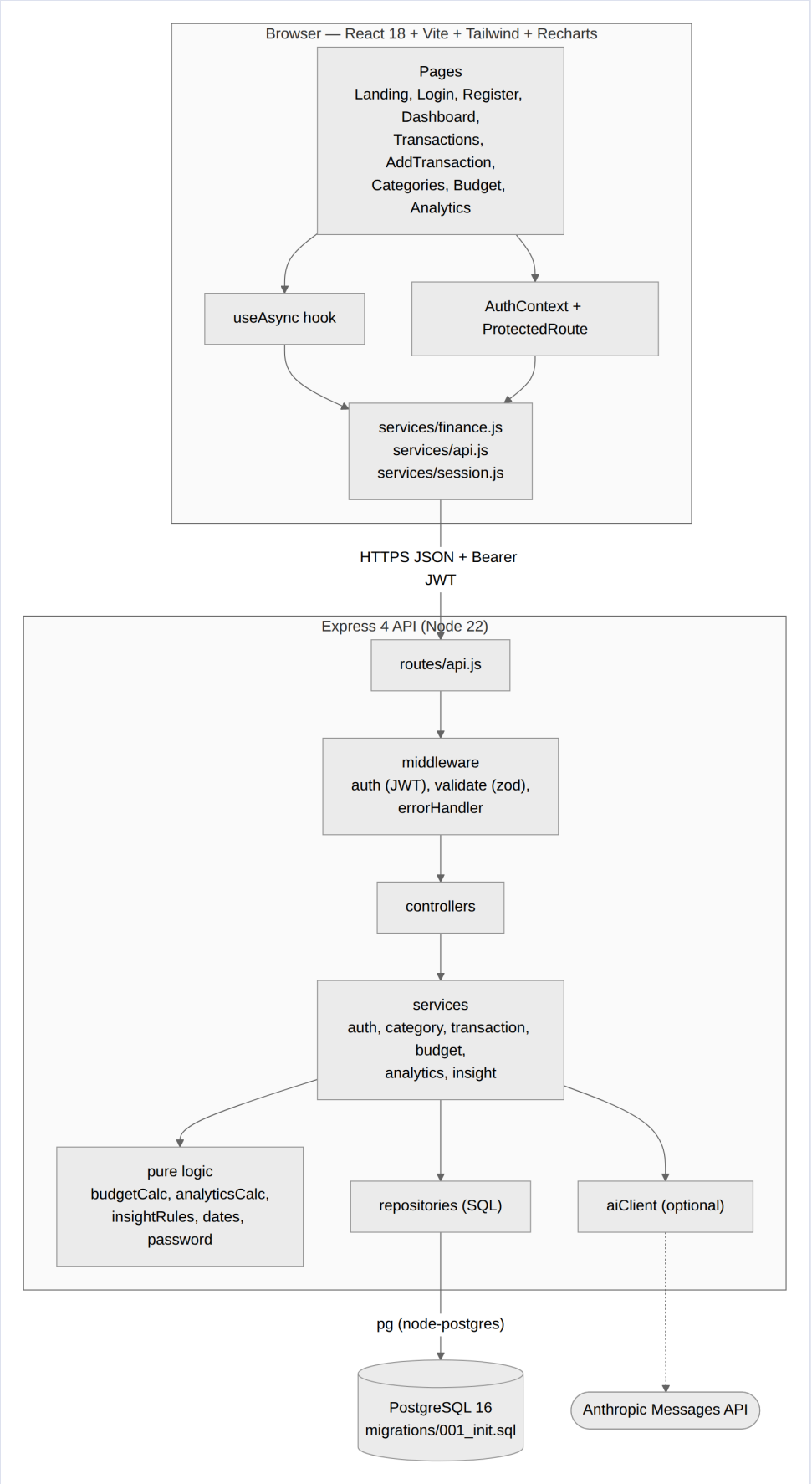


Figure D.2 — Component diagram.

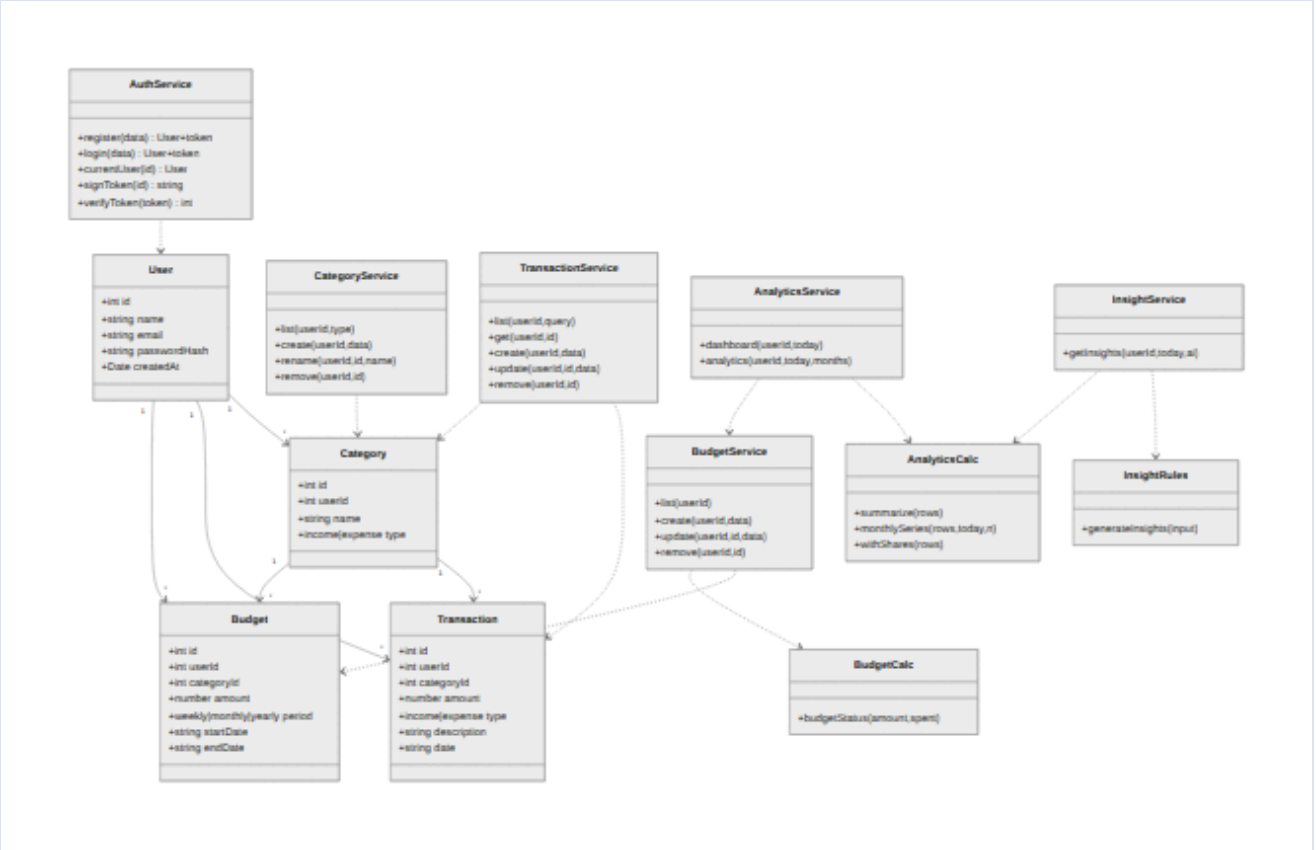


Figure D.3 — Class diagram (entities, services and pure calculation modules).

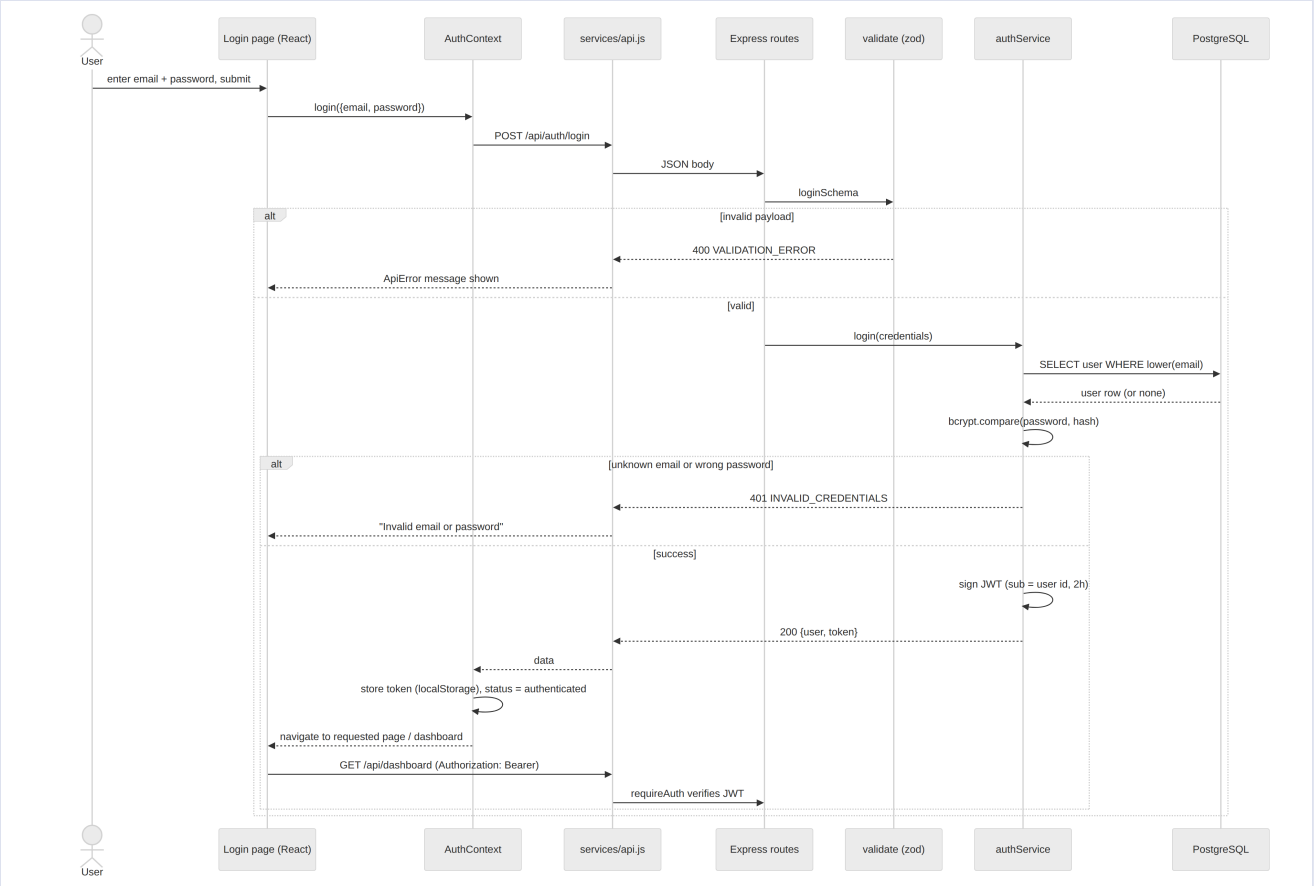


Figure D.4 — Sequence diagram: login.

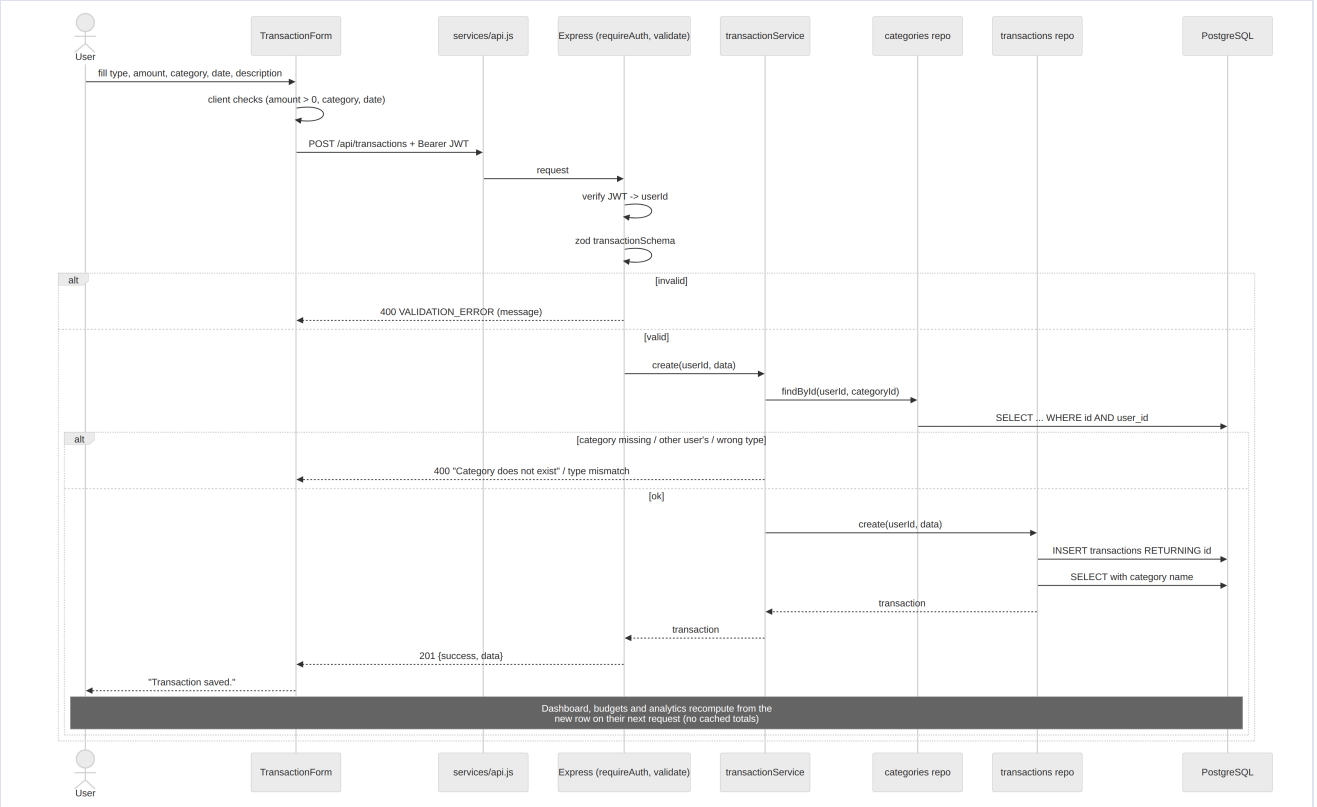


Figure D.5 — Sequence diagram: adding a transaction.

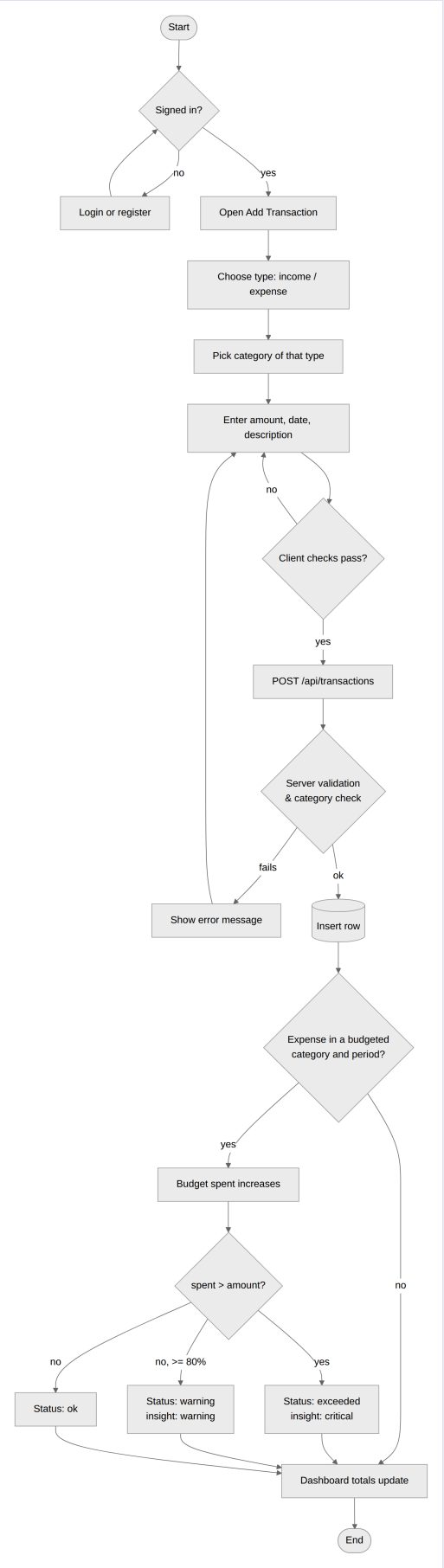


Figure D.6 — Activity diagram: recording a transaction and its effect on budgets.

E. Code Listing / GitHub Link

Full source: **https://github.com/vehladkid/FInEdge** (release tag `v1.0.0`). Representative listings follow; they are extracted from the repository at build time.

`backend/src/services/budgetCalc.js` **— budget status calculation (lines 1–20)**

```
import { round2 } from "../utils/money.js";

// Pure budget arithmetic. "warning" starts at 80% used; "exceeded" is strictly
// over the allocation (spending exactly the amount is 100% but not exceeded).
export const WARNING_THRESHOLD = 80;

export function budgetStatus(amount, spent) {
  const remaining = round2(amount - spent);
  const percentUsed = amount > 0 ? Math.round((spent / amount) * 1000) / 10 : 0;
  const exceeded = spent > amount;
  let status = "ok";
  if (exceeded) status = "exceeded";
  else if (percentUsed >= WARNING_THRESHOLD) status = "warning";
  return { spent: round2(spent), remaining, percentUsed, exceeded, status };
}
```

`backend/src/repositories/budgets.js` **— budget spend computed by the database (lines 1–17)**

```
import { query } from "../db/pool.js";

// `spent` is the sum of expense transactions in the budget's category inside
// [start_date, end_date] — computed by the database, never trusted from clients.
const SELECT = `
  SELECT b.id, b.amount, b.period, b.start_date AS "startDate", b.end_date AS "endDate",
         b.category_id AS "categoryId", c.name AS category,
         COALESCE((SELECT sum(t.amount) FROM transactions t
                   WHERE t.user_id = b.user_id AND t.category_id = b.category_id
                     AND t.type = 'expense'
                     AND t.transaction_date BETWEEN b.start_date AND b.end_date), 0) AS spent
  FROM budgets b JOIN categories c ON c.id = b.category_id`;

export async function list(userId) {
  const { rows } = await query(`${SELECT} WHERE b.user_id = $1 ORDER BY b.start_date DESC, c.name`,
[userId]);
  return rows;
}
```

`backend/src/services/authService.js` **— registration and login (lines 22–56)**

```
export async function register({ name, email, password }) {
  if (await users.findByEmail(email)) throw conflict("An account with this email already exists");
  const passwordHash = await bcrypt.hash(password, config.bcryptRounds);
  const client = await pool.connect();
  try {
    await client.query("BEGIN");
    const user = await users.create({ name, email, passwordHash }, client);
    for (const c of DEFAULT_CATEGORIES) await categories.create(user.id, c, client);
    await client.query("COMMIT");
    return { user, token: signToken(user.id) };
  } catch (err) {
    await client.query("ROLLBACK");
    // Two simultaneous registrations can both pass the check above.
    if (err.code === "23505") throw conflict("An account with this email already exists");
    throw err;
  } finally {
    client.release();
  }
}

const INVALID = "Invalid email or password";

export async function login({ email, password }) {
  const user = await users.findByEmail(email);
  const ok = user ? await bcrypt.compare(password, user.passwordHash) : false;
  if (!user || !ok) throw new AppError(401, "INVALID_CREDENTIALS", INVALID);
  const { passwordHash, ...publicUser } = user; // eslint-disable-line no-unused-vars
  return { user: publicUser, token: signToken(user.id) };
}

export async function currentUser(userId) {
  const user = await users.findById(userId);
  if (!user) throw unauthorized("Account no longer exists");
  return user;
}
```

`backend/src/middleware/auth.js` — JWT guard (lines 1–15)

```
import { unauthorized } from "../errors/index.js";
import { verifyToken } from "../services/authService.js";

// Expects "Authorization: Bearer <jwt>"; sets req.userId.
export function requireAuth(req, _res, next) {
  const header = req.headers.authorization ?? "";
  const [scheme, token] = header.split(" ");
  if (scheme !== "Bearer" || !token) return next(unauthorized());
  try {
    req.userId = verifyToken(token);
    next();
  } catch (err) {
    next(err);
  }
}
```

`backend/tests/integration/budgets.test.js` — integration test — budget boundaries (lines 63–82)

```
    const other = (await api.post("/api/budgets", budget({ period: "weekly" }))).body.data;
    const res = await api.put(`/api/budgets/${other.id}`, { amount: 5, period: "monthly", startDate:
monthStart });
    expect(res.status).toBe(409);
  });
});

describe("budget calculations come from real transactions", () => {
  it("reports spent, remaining and percentage", async () => {
    const { id } = (await api.post("/api/budgets", budget())).body.data;
    await spend(100);
    await spend(140.5);
    const b = (await api.get(`/api/budgets/${id}`)).body.data;
    expect(b).toMatchObject({ spent: 240.5, remaining: 159.5, percentUsed: 60.1, exceeded: false,
status: "ok" });
  });

  it("warns at 80% and treats exactly-at-budget as not exceeded", async () => {
    const { id } = (await api.post("/api/budgets", budget())).body.data;
    await spend(320);
    expect((await api.get(`/api/budgets/${id}`)).body.data).toMatchObject({ percentUsed: 80, status:
"warning" });
    await spend(80);
```